UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

Worksheet 5

Student Name:Aryan Tomar **UID:25MCI10167**
Branch:MCA (AI&ML) **Section/Group:1/A**
Semester:2nd **Date of Performance:01/03/2026**
Subject Name:- DBMS LAB **Subject Code: 25CAP-652**

1. Aim of the Session

To gain hands-on experience in creating and using cursors for row-by-row processing in a database, enabling sequential access and manipulation of query results for complex business logic. **(Company Tags: Infosys, Wipro, TCS, Capgemini)**

2. Software Requirements

- PostgreSQL (Database Server)
- pgAdmin
- Windows Operating System

3. Objective of the Session

- **Sequential Data Access:** To understand how to fetch rows one by one from a result set using cursor mechanisms.
- **Row-Level Manipulation:** To perform specific operations or calculations on individual records that require conditional procedural logic.
- **Resource Management:** To learn the lifecycle of a cursor: Declaring, Opening, Fetching, and importantly, Closing and Deallocating to manage system memory.
- **Exception Handling:** To handle cursor-related errors and performance considerations during large-scale data iteration.

4. Practical / Experiment Steps

Prerequisite Understanding

• Students should first understand that cursors are used inside PL/pgSQL blocks for row-by-row processing, not in normal set-based SQL queries.

• Students should understand the difference between set-oriented SQL execution and procedural row-level execution.

• Students should create a table containing multiple records so that sequential row processing using cursors can be demonstrated.

• The table should contain:
  • A unique identifier (Primary Key)
  • A descriptive attribute (such as employee name or department)
  • A numeric attribute (such as salary or experience) to apply conditional logic

• Students should understand the lifecycle of a cursor, including:
  • DECLARE – defining the cursor query
  • OPEN – executing the cursor
  • FETCH – retrieving rows sequentially
  • CLOSE – releasing the result set

• Students should be familiar with basic IF–ELSE logic for applying business rules during row processing.

5. Procedure of the Practical

(i) Start the system and log in to the computer.

(ii) Open PostgreSQL software.

iii) Create and select the database.

Create database Practical5;

(iv) Create table using DDL command.

```
   emp_id serial primary key,
   emp_name varchar(50),
   experience int,
   salary numeric(10,2)
);
```

(v) Insert records into the table.

```
insert into employee_cursor (emp_name, experience, salary) values
('Rahul', 2, 30000),
('Ankit', 5, 45000),
('Priya', 8, 60000),
('Neha', 1, 25000),
('Aman', 10, 80000);
```

(vi) Display all records.

```
select * from employee_cursor;
```

	emp_id [PK] integer	emp_name character varying (50)	experience integer	salary numeric (10,2)
1	1	Rahul	2	30000.00
2	2	Ankit	5	45000.00
3	3	Priya	8	60000.00
4	4	Neha	1	25000.00
5	5	Aman	10	80000.00

Step 1: Implementing a Simple Forward-Only Cursor

```
do $$
declare
    emp_rec record;
    emp_cursor cursor for
        select emp_id, emp_name, salary from employee_cursor;
begin
    open emp_cursor;

    loop
        fetch emp_cursor into emp_rec;
        exit when not found;
```

```
        raise notice 'ID: %, Name: %, Salary: %',
        emp_rec.emp_id, emp_rec.emp_name, emp_rec.salary;
    end loop;

    close emp_cursor;
end $$;
```

```
NOTICE:  ID: 1, Name: Rahul, Salary: 30000.00
NOTICE:  ID: 2, Name: Ankit, Salary: 45000.00
NOTICE:  ID: 3, Name: Priya, Salary: 60000.00
NOTICE:  ID: 4, Name: Neha, Salary: 25000.00
NOTICE:  ID: 5, Name: Aman, Salary: 80000.00
DO

Query returned successfully in 160 msec.
```

Step 2: Complex Row-by-Row Manipulation

```
do $$
declare
    emp_rec record;
    emp_cursor cursor for
        select emp_id, experience, salary from employee_cursor;
begin
    open emp_cursor;
```

```
    loop
        fetch emp_cursor into emp_rec;
        exit when not found;

        if emp_rec.experience >= 8 then
            update employee_cursor
            set salary = salary * 1.20
            where emp_id = emp_rec.emp_id;

        elseif emp_rec.experience >= 5 then
            update employee_cursor
            set salary = salary * 1.10
            where emp_id = emp_rec.emp_id;

        else
            update employee_cursor
            set salary = salary * 1.05
            where emp_id = emp_rec.emp_id;
        end if;

    end loop;

    close emp_cursor;
end $$;
```

	emp_id [PK] integer	emp_name character varying (50)	experience integer	salary numeric (10,2)
1	1	Rahul	2	31500.00
2	2	Ankit	5	49500.00
3	3	Priya	8	72000.00
4	4	Neha	1	26250.00
5	5	Aman	10	96000.00

Step 3: Exception and Status Handling

```
do $$
declare
    emp_rec record;
    emp_cursor cursor for
        select * from employee_cursor where experience > 20;
begin
    open emp_cursor;

    fetch emp_cursor into emp_rec;

    if not found then
        raise notice 'No employees found with experience > 20.';
    else
        raise notice 'Employees exist.';
```

```
    end if;

    close emp_cursor;
end $$;
```

```
NOTICE:  No employees found with experience > 20.
DO

Query returned successfully in 102 msec.
```

6. I/O Analysis (Input / Output)

Input:

- Employee records inserted into a database table for row-by-row processing
- PL/pgSQL DO blocks containing cursor logic
- DECLARE, OPEN, FETCH, and CLOSE cursor statements
- Conditional IF–ELSE logic applied within cursor loops
- UPDATE statements executed based on experience or salary conditions
- Exception handling logic for empty result sets

Output:

- Sequential row-by-row processing of employee records using cursor mechanism
- Display of employee details using FETCH operation
- Conditional salary updates based on defined business rules
- Notice messages generated during cursor execution
- Proper termination of cursor after processing all records

• Successful handling of empty result sets without runtime errors

• Correct execution of procedural SQL demonstrating cursor lifecycle

Screenshots of execution and obtained results are attached.

7. Learning Outcomes

- **Cursor Implementation:** Students will be able to design, implement, and manage cursors to solve row-wise processing problems.
- **Lifecycle Mastery:** Students will demonstrate the correct syntax for declaring, opening, fetching, and closing cursors.
- **Error Prevention:** Students will understand how to properly handle row-by-row processing exceptions and prevent memory leaks via deallocation.
- **Analytical Thinking:** Students will be able to apply cursor-based logic to solve real-world scenarios like multi-level payroll adjustments or data migrations.